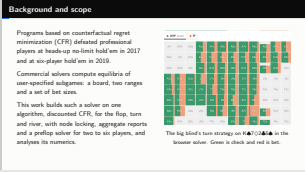
Range-Vectorized Discounted Regret Minimization for No-Limit Hold'em:
Complexity, Precision and Parallelism

Numerical Analysis Research Club

Timing for 60 minutes. Problem and equilibria 7 · CFR and DCFR 7 · Traversal and leaves 9 · Cost, memory, precision and parallelism 9 · Convergence and validation 6 · Demonstration 9 · Extensions and next steps 8 · Questions 5.

Opening line: "A poker solver is one numerical method, discounted counterfactual regret minimization, applied to a structured bilinear saddle-point problem. This talk is about what makes that method fast and accurate: how an iteration is organized, what it costs, how precise it must be and how it is split across processors."

Every measured number comes from the experiment scripts listed in `web/experiments/run_all.sh`, with logs in `out/`, through `tex/make_talk_data.py`. The machine is a two-core cloud container, so absolute times are pessimistic.



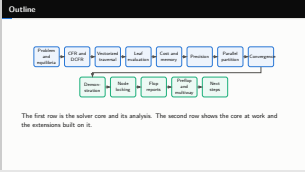
Libratus and DeepStack (2017) and Pluribus (2019) are the milestones, and their inner loop is a member of the CFR family. The feature list of this project follows the commercial tools PioSolver and MonkerSolver: flop, turn and river solving, arbitrary bet trees, rake, node locking, aggregate flop reports, preflop and multiway solving.

On the algorithm side the scope is narrow on purpose: one method and its numerical analysis. New algorithms are on the next-steps slide.

Range-Vectorized Discounted Regret Minimization for No-Limit Hold'em: Complexity, Precision and Parallelism

└Outline

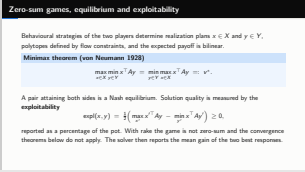
Keep this slide short.





The running example is the most common spot in online poker: the button opens, the big blind calls, and the flop is K♠ 7♢ 2♣. The big blind has 488 hands and the button 460 that avoid the board.

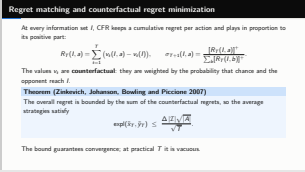
The table gives the spots of the convergence study, one tree per street. A river spot has a few dozen decision nodes; a flop spot with one bet size per street has tens of thousands, because every turn and river card opens a new copy of the later streets.



This is a convex-concave saddle-point problem with bilinear coupling, and the exploitability is the primal-dual gap. Commercial solvers stop between 0.1% and 0.5% of the pot.

The sequence-form linear program of Koller, Megiddo and von Stengel solves the game exactly, but its matrix has a dense hand-by-hand block at every leaf; a flop tree has about seventy thousand leaves with blocks of size about 500×500. Iterative methods that only need products with A and $A^{\top}$ are the only option, and those products never need A either.

With rake, DCFR is used as a heuristic; the measured exploitability is still reported and in our runs it decreases in the same way.

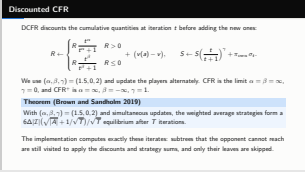


For the flop spot $|\mathcal{I}| \approx 2.7 \times 10^7$, so the bound exceeds the pot by many orders of magnitude at any feasible T.

Regret matching (Hart and Mas-Colell 2000) keeps the maximal regret of order $\sqrt{T}$ by Blackwell approachability: the instantaneous regret is orthogonal to the current positive regret vector, so the squared norm of the positive regrets grows at most linearly.

CFR applies regret matching at each information set with the counterfactual value as payoff. The decomposition theorem bounds the global regret by the sum of these local regrets, and average regret ε for both players gives an ε-equilibrium of the averages in our normalization.

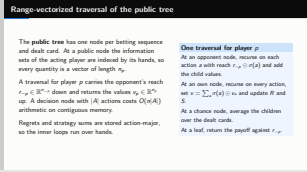
Δ is the payoff range, $|\mathcal{I}|$ the number of information sets (hands times decision nodes) and $|A|$ the largest number of actions. With $\Delta \approx 2 \cdot 97.5$ chips, $|\mathcal{I}| \approx 2.7 \times 10^7$ and $|A| = 3$, the bound at $T = 10^4$ is above 10^7 chips. The observed behaviour is far better and is measured later.



The theorem is proved for simultaneous updates. The solver alternates the players, which is standard practice; regret bounds for alternating updates are discussed by Burch, Moravčík and Schmid (2019).

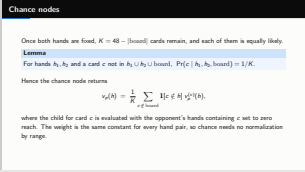
Discounting is a forgetting factor. Positive regrets lose a fraction $1/(t^{1.5}+1)$, negative regrets are halved, and the average gives weight proportional to t^2 to iteration t. Alternating updates are Gauss–Seidel in place of Jacobi.

Exactness matters. An earlier version skipped subtrees with zero opponent reach entirely, which also skipped their discounts and strategy-sum updates. On a small flop game this left the exploitability at 0.64 chips after 1000 iterations instead of 0.0025, a factor of 250. A test now checks that the regrets and sums agree bit for bit with a traversal that never skips. The extra cost is the decision work of the skipped subtrees; their leaves are still skipped.



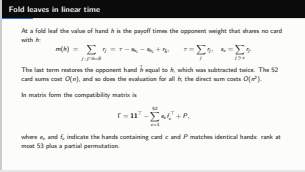
A naive implementation traverses the tree once per hand pair. The vectorized form traverses it once per player and iteration, with vector arithmetic of length about 500 at each node.

The products Ay and $A^\top x$ of the earlier slide are exactly the values returned to the root. The opponent's reach plays the role of y, and the payoff blocks are applied at the leaves without being stored. All buffers for one depth are allocated once, so a traversal allocates no memory.



The conditional probability of the next card depends only on the four known private cards, which are excluded by the indicator and by zeroing the opponent's reach. Using $1/(52 - |\text{board}|)$ would bias every value.

The lemma is checked against exact linear programs with explicit chance nodes, one chance level on the turn and two on the flop (validation slide).


Fold leaves in linear time

At a fold leaf the value of hand h is the payoff times the opponent weight that shares no card with h:

$$m(h) = \sum_{j:\, j^{\mathrm{rank}} \cap h = \emptyset} r_j = \tau - a_{h^{\mathrm{rank}}} - a_{h^{\mathrm{rank}}} + \tau_{h^{\mathrm{rank}}}, \qquad \tau = \sum_j r_j, \quad a_v = \sum_{j \ni v} r_j.$$

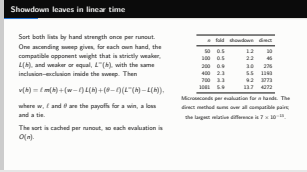
The last term restores the opponent hand $\hat{h}$ equal to h, which was subtracted twice. The 52 card sums cost $O(n)$, and so does the evaluation for all h; the direct sum costs $O(n^2)$.

In matrix form the compatibility matrix is

$$\Gamma = \mathbf{1}\mathbf{1}^{\top} - \sum_{c=1}^{52} e_c e_c^{\top} + P,$$

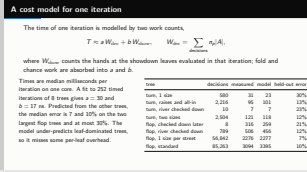
where e_c and ℓ_c indicate the hands containing card c and P matches identical hands: rank at most 53 plus a partial permutation.

Inclusion–exclusion over the two cards of h. Two different two-card hands share at most one card, except the identical hand, which shares both and is subtracted twice. This is the card-removal effect that players talk about, computed exactly in one pass over each list (Johanson and coauthors 2011).



After sorting, the sign matrix of the showdown is lower triangular ones minus upper triangular ones with tie blocks on the diagonal: a semiseparable matrix, so its product with a vector is a prefix sum. The compatibility mask is handled by per-card sums updated inside the sweep.

At 1081 hands the sweep is several hundred times faster than the direct sum and agrees to rounding. A flop iteration evaluates about 42,336 showdown leaves per traversal.


A cost model for one iteration

The time of one iteration is modelled by two work counts,

$$T \approx a\,W_{\text{dec}} + b\,W_{\text{showdown}}, \qquad W_{\text{dec}} = \sum_{\text{decisions}} n_p |A|,$$

where W_{showdown} counts the hands at the showdown leaves evaluated in that iteration; fold and chance work are absorbed into a and b.

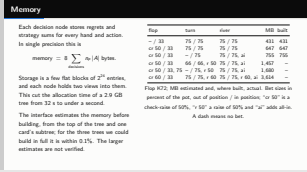
Times are median milliseconds per iteration on one core. A fit to 252 timed iterations of 8 trees gives $a \simeq 30$ and $b \simeq 17$ ns. Predicted from the other trees, the median error is 7 and 10% on the two largest flop trees and at most 30%. The model under-predicts leaf-dominated trees, so it misses some per-leaf overhead.

tree	decisions	measured	model	held-out error
turn, 1 size	580	31	23	30%
turn, raises and all-in	2,216	95	101	13%
turn, river checked down	10	7	7	20%
turn, two sizes	2,504	121	118	12%
flop, checked down later	8	316	259	21%
flop, river checked down	789	506	456	12%
flop, 1 size per street	56,842	2276	2277	7%
flop, standard	85,263	3094	3395	10%

Each iteration is timed together with the work counted in the same iteration, over the first 14 to 60 iterations of a solve; the first two iterations of each tree (compilation, first sorts) are left out. The cost depends on the iterate, because subtrees that the opponent does not reach skip their leaves, and individual iterations of one tree vary by up to 72% (garbage collection, compilation and the changing reach). The model accounts for the time; it does not predict it in advance, since W_{show} is measured by running.

Fitting separate fold, chance and decision constants is ill-posed because these counts are nearly collinear across trees, so a includes the fold and chance work. On the flop tree the showdowns take about 29% of the time. The bare DCFR update loop in the same runtime costs 3.3 ns per entry; the remainder of a is the rest of the traversal (regret matching, reach products, folds, chance nodes, recursion), not memory bandwidth, which the runtime delivers at 5.4 GB/s.

Adding a bet size multiplies the tree, because it adds a branch at every node of the street and a copy of all later streets below it.



Memory

Each decision node stores regrets and strategy sums for every hand and action. In single precision this is

$$\text{memory} \approx 8 \sum_{\text{decision}} n_{\mathcal{P}} |A| \text{ bytes.}$$

Storage is a few flat blocks of 2^{25} entries, and each node holds two views into them. This cut the allocation time of a 2.9 GB tree from 32 s to under a second.

The interface estimates the memory before building, from the top of the tree and one card's subtree; for the three trees we could build in full it is within 0.1%. The larger estimates are not verified.

flop	turn	river	MB	built
— / 33	75 / 75	75 / 75	411	411
cr 50 / 33	75 / 75	75 / 75	647	647
cr 50 / 33	— / 75	75 / 75, ai	755	756
cr 50 / 33	66 / 66, r 50	75 / 75, ai	1,457	–
cr 50 / 33, 75	— / 75, r 50	75 / 75, ai	1,668	–
cr 66 / 33	75 / 75, r 40	75 / 75, r 60, ai	3,644	–

Flop K72; MB estimated and, where built, actual. Bet sizes in percent of the pot, out of position / in position. "cr 50" is a check-raise of 50%, "r 50" a raise of 50% and "ai" adds all-in. A dash means no bet.

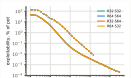
Memory is the binding constraint in a browser. One typed array per node made the allocator the bottleneck; flat blocks make allocation and garbage collection cheap and keep each node's data contiguous.

Below a chance node the subtrees for different cards have the same shape and differ only in the hands, so one card's subtree times the number of cards is an accurate estimate.

Floating-point precision

Regrets R and strategy sums S are stored in single precision; all arithmetic inside a traversal is double precision.

All four combinations of single and double precision for R and S reach the same exploitability, down to 2×10^{-4}% of the pot. The strategies differ by a reach-weighted L^1 distance of 6.8×10^{-7} on the small game and 8.1×10^{-7} on the turn spot. Double precision runs whose ranges differ by 10^{-7} differ just as much (9.012 and 7.5×10^{-7}); so this is the sensitivity of the DCFR trajectory, not storage precision.



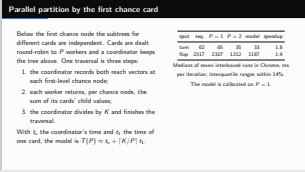
Exploitability with float32 or float64 storage of R and S, on a small turn game (solid) and the turn spot (dashed).

Each stored update has relative rounding error $u = 2^{-24}$. The strategy sum with DCFR weights grows like $t/3$, so the new term has relative size about $3/t$ and is resolved while $t \ll 10^7$. The strategy is a ratio of sums, so relative errors of the sums enter the strategy as relative errors of size u per update. A term is lost only when its relative size falls below u, which needs about 10^7 iterations.

Any perturbation of relative size 10^{-7}, whether rounding or a change in the inputs, leads DCFR to a different approximate equilibrium of the same exploitability. Two iterates of one run, at $T/1.1$ and T, are much closer (1.2×10^{-4} and 5.5×10^{-3}). On the small game the distance between the perturbed runs stays near 0.013 while the exploitability falls from 0.047 to 2.0×10^{-4}% of the pot: the set of near-equilibria is wide, and the strategy is ill-conditioned with respect to exploitability. Trajectory sensitivity is the consequence. The exploitability, which is what a solver guarantees, is not affected. An earlier version of the solver seemed to show a single-precision floor near 10^{-3}% of the pot; it disappeared with the exact zero-reach handling of the DCFR slide, so it was an artefact of the iteration, not of rounding.

On the turn spot the curves group by the precision of the regrets: the precision of the strategy sums makes no visible difference.

Half the memory for the same result.


Parallel partition by the first chance card

Below the first chance node the subtrees for different cards are independent. Cards are dealt round-robin to P workers and a coordinator keeps the tree above. One traversal is three steps:

1. the coordinator records both reach vectors at each first-level chance node;
2. each worker returns, per chance node, the sum of its cards' child values;
3. the coordinator divides by K and finishes the traversal.

With t_c the coordinator's time and t_1 the time of one card, the model is $T(P) \approx t_c + \lceil K/P \rceil\, t_1$.

spot	seq.	$P=1$	$P=2$	model	speedup
turn	62	65	35	33	1.8
flop	2317	2327	1212	1167	1.9

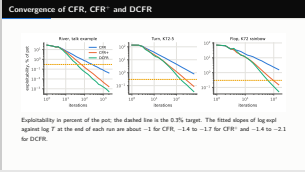
Medians of seven interleaved runs in Chrome, ms per iteration; interquartile ranges within 14%.

The model is calibrated on $P = 1$.

This is a domain decomposition of the tree with the first-level chance nodes as interface. The messages are two vectors of length n per chance node and worker, at most 44 kB per traversal, against millions of operations per card. The coordinator's share is under a millisecond.

The results differ from the sequential solver only in the order in which the chance sums are added; they are not bitwise identical. On the flop, two workers reach 98% of the model's prediction, a difference within the run-to-run spread. The model ignores message latency and the competition of the two workers for memory bandwidth and cache, and the main thread also runs the page. The model predicts 333 ms with eight workers. On the turn the passes are short, so the latency matters more.

In Node's worker threads the same partitioned traversal runs about 48% slower than on the main thread for reasons we have not identified; in Chrome the one-worker overhead is under 1% on the flop and 5% on the turn, where the passes are short. The eight-worker figure is an extrapolation of the model, not a measurement. In the browser the workers are Web Workers and the coordinator is the main thread. Games without a chance node, river spots, run in one worker.

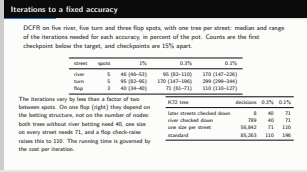


Exploitability in percent of the pot; the dashed line is the 0.3% target. The fitted slopes of log expl against log T at the end of each run are about -1 for CFR, -1.4 to -1.7 for CFR$^+$ and -1.4 to -2.1 for DCFR.

All three methods use the same code and differ only in (α, β, γ). DCFR is fastest on every street. Slopes over the last twentieth, tenth and fifth of each run:

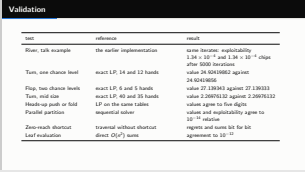
	river	turn	flop
CFR	-1.0 to -1.0	-1.0 to -1.0	-1.0 to -0.9
CFR$^+$	-1.6 to -1.5	-1.5 to -1.4	-1.7 to -1.6
DCFR	-1.5 to -1.4	-1.7 to -1.6	-2.1 to -2.1

The slopes are empirical rates over the end of each run and depend on the window, as the ranges show; the flop windows still include part of the early transient. They should not be read as asymptotic rates; the worst-case bound is $T^{-1/2}$ for all three. Explaining why the modern variants do so much better on poker trees is an open problem.



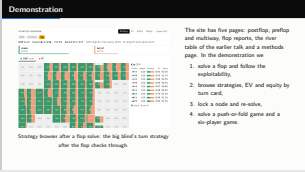
Boards: river K72T4, A8492 two-tone, T9832, QJ552, 762KA; the turn spots are the first four cards of the same boards; flop spots K72 rainbow, T98 two-tone and A84 two-tone. Ranges are the big blind's call and the button's open.

This is the practical statement behind "a few hundred iterations": it is a measured median over spots, not a theorem.



Most components have an independent check. The two LP tests build the game separately in Python with explicit chance nodes and solve the sequence-form LP exactly, so they test chance weighting, card removal and showdowns together; the remaining differences, 6×10^{-8} and 1×10^{-5}, are within the exploitability left after 20,000 and 10,000 DCFR iterations. The river row checks the new engine against the earlier implementation of the same algorithm: equal iterates, not an independent method. The mid-size turn LP, with 40 and 35 hands drawn at random, exercises card removal among overlapping hands, ties and chance weighting; its difference 7×10^{-9} is within the remaining exploitability 5×10^{-6}. Values are compared rather than strategies, because equilibria are not unique.

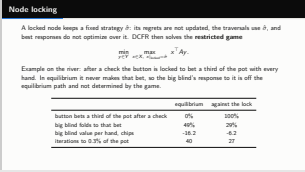
A comparison with an established open-source solver on the same flop tree is the natural next test.



Strategy browser after a flop solve: the big blind's turn strategy after the flop checks through.

Switch to the browser. The script is app/DEMO_NOTES.md; budget nine minutes.

(1) Postflop page, example spot K72, one size per street. Build and solve; point at the memory estimate, the worker count and the exploitability curve with the 0.3% target. (2) The big blind has no flop bet in this tree, so the first decision is the button's: click check, then the five of spades. Show the grid, then EV and equity, and point at one square. (3) Node locking: edit, choose the bet, paint a row, lock, continue. (4) Preflop page, heads-up push or fold at 10 bb: 58% shove and 37% call. Then the six-max preset and the first-in table.



Node locking

A locked node keeps a fixed strategy $\hat{\sigma}$: its regrets are not updated, the traversals use $\hat{\sigma}$, and best responses do not optimize over it. DCFR then solves the **restricted game**

$$\min_{x \in \mathcal{X}} \max_{y \in \mathcal{Y},\, y_{\text{locked}} = \hat{\sigma}} x^\top A y.$$

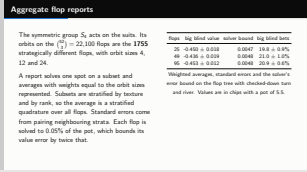
Example on the river: after a check the button is locked to bet a third of the pot with every hand. In equilibrium it never makes that bet, so the big blind's response to it is off the equilibrium path and not determined by the game.

	equilibrium	against the lock
button bets a third of the pot after a check	0%	100%
big blind folds to that bet	46%	29%
big blind value per hand, chips	-16.2	-6.2
iterations to 0.3% of the pot	40	27

Locking answers the practical question: what is the best response to a known mistake, while every other node also adapts.

Numerically, the restricted game is no harder: solved from scratch, it reaches 0.3% of the pot in about as many iterations as the free game.

On flop spots a node below a chance card is locked for that card only, which matches commercial tools.


Aggregate flop reports

The symmetric group S_4 acts on the suits. Its orbits on the $\binom{52}{3} = 22{,}100$ flops are the **1755** strategically different flops, with orbit sizes 4, 12 and 24.

A report solves one spot on a subset and averages with weights equal to the orbit sizes represented. Subsets are stratified by texture and by rank, so the average is a stratified quadrature over all flops. Standard errors come from pairing neighbouring strata. Each flop is solved to 0.05% of the pot, which bounds its value error by twice that.

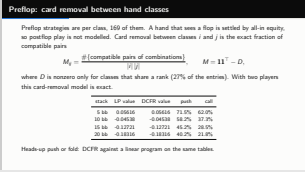
flops	big blind value	solver bound	big blind bets
35	-0.450 ± 0.018	0.0047	19.8 ± 0.9%
84	-0.438 ± 0.019	0.0043	21.0 ± 1.0%
95	-0.453 ± 0.012	0.0048	20.9 ± 0.6%

Weighted average, standard errors and the solver's error bound on the flop tree with checked-down turn and river. Values are in chips with a pot of 5.5.

Burnside's lemma gives 1755 (appendix). Orbit sizes are 24 for rainbow boards, 12 for two-tone and paired boards and 4 for monotone and trips boards.

The value of the average strategy differs from the equilibrium value by at most the larger best-response gain, hence by at most twice the exploitability; the table gives this bound averaged over the flops. Bet frequencies have no such bound, which is why the flops are solved to a tight tolerance.

The report is a quadrature of a function of the flop. The collapsed-strata variance estimator pairs neighbouring strata and treats the pair as one stratum with two draws, which overestimates the variance slightly. The estimates agree within their standard errors as the subset grows.


Preflop: card removal between hand classes

Preflop strategies are per class, 169 of them. A hand that sees a flop is settled by all-in equity, so postflop play is not modelled. Card removal between classes i and j is the exact fraction of compatible pairs

$$M_{ij} = \frac{\#\{\text{compatible pairs of combinations}\}}{|i|\,|j|}, \qquad M = \mathbf{1}\mathbf{1}^{\top} - D,$$

where D is nonzero only for classes that share a rank (27% of the entries). With two players this card-removal model is exact.

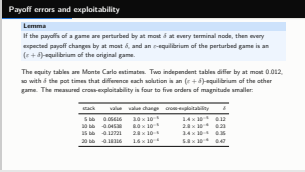
stack	LP value	DCFR value	push	call
5 bb	0.05616	0.05616	71.5%	62.0%
10 bb	−0.04538	−0.04538	58.2%	37.3%
15 bb	−0.12721	−0.12721	45.2%	26.5%
20 bb	−0.18316	−0.18316	40.2%	21.8%

Heads-up push or fold. DCFR against a linear program on the same tables.

The equity tables are computed once offline: 169 by 169 class equities by Monte Carlo with 50,000 deals per pair, and exact pair counts.

The product Mr costs 169^2 dense, but with D sparse it is a sum minus a sparse correction, about three times cheaper. Products are cached per reach vector, because a vector formed at one node is shared by every leaf below it.

The LP has 169 strategy variables and 169 constraints. At 10 bb the frequencies are the published Nash push-or-fold numbers.


Payoff errors and exploitability

Lemma

If the payoffs of a game are perturbed by at most δ at every terminal node, then every expected payoff changes by at most δ, and an ε-equilibrium of the perturbed game is an $(\varepsilon + \delta)$-equilibrium of the original game.

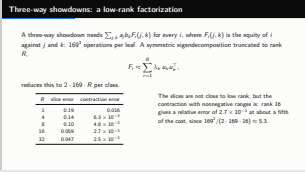
The equity tables are Monte Carlo estimates. Two independent tables differ by at most 0.012, so with δ the pot times that difference each solution is an $(\varepsilon + \delta)$-equilibrium of the other game. The measured cross-exploitability is four to five orders of magnitude smaller:

stack	value	value change	cross-exploitability	δ
5 bb	0.06616	3.0×10^{-5}	1.4×10^{-5}	0.12
10 bb	−0.04538	8.0×10^{-5}	2.8×10^{-5}	0.23
15 bb	−0.12721	2.8×10^{-4}	3.4×10^{-5}	0.35
20 bb	−0.18316	1.6×10^{-4}	5.9×10^{-6}	0.47

Proof of the lemma: each best-response value is a maximum of expected payoffs, each of which moves by at most δ, so each half of the exploitability moves by at most $\delta/2$ in our normalization.

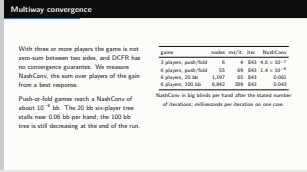
The two tables define two games whose payoffs differ by at most δ, so one application of the lemma gives the bound $\varepsilon + \delta$ rigorously. Cross-exploitability is the exploitability of the table-1 solution measured in the table-2 game. It is four to five orders of magnitude below the bound, because errors of different entries have different signs and average out over a range. The push frequencies of the two solutions differ by at most 0.4 percentage points; the lemma says nothing about strategies, which need not be unique.

The accuracy against the true equities is a separate question: the Monte Carlo standard error of each entry is about 0.2%. The low-rank approximation of the next slide is a further perturbation; its contraction error is an empirical estimate over random ranges, not a worst-case δ.

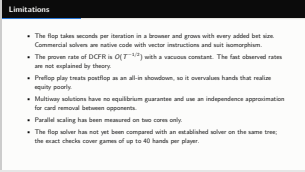


Errors are the largest over the 169 classes. The slice error is the relative Frobenius error of F_i; the contraction error is the largest relative error of $a^\top F_i b$ over 20 pairs of random ranges per class, with entries uniform on $[0, 1]$ and each kept with probability 0.4.

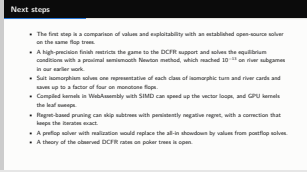
The tensor has 4.8 million entries, estimated with 1000 deals per sorted triple.



This is consistent with the literature: Pluribus used CFR-type self-play for six players without a guarantee, and it worked well in practice. With three players in a pot, opponents' cards are treated as independent given the hero's hand, which ignores card removal between the opponents.



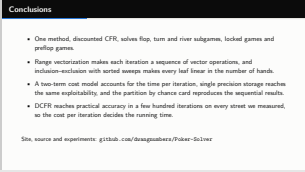
The browser engine is plain JavaScript; the next slide lists the steps that close most of the gap to native solvers. The preflop limitation is the largest modelling simplification: MonkerSolver solves the postflop as well, and simpler tools use realization factors.


Next steps

- The first step is a comparison of values and exploitability with an established open-source solver on the same flop trees.
- A high-precision finish restricts the game to the DCFR support and solves the equilibrium conditions with a proximal semismooth Newton method, which reached 10^{-13} on river subgames in our earlier work.
- Suit isomorphism solves one representative of each class of isomorphic turn and river cards and saves up to a factor of four on monotone flops.
- Compiled kernels in WebAssembly with SIMD can speed up the vector loops, and GPU kernels the leaf sweeps.
- Regret-based pruning can skip subtrees with persistently negative regret, with a correction that keeps the iterates exact.
- A preflop solver with realization would replace the all-in showdown by values from postflop solves.
- A theory of the observed DCFR rates on poker trees is open.

The second item is the research continuation (`talk-newton.pdf`): from a DCFR start, a proximal Newton method with interior-point crossover finished small and medium river games to machine precision. Extending it to turn and flop trees needs the structured linear algebra of the leaves inside the Newton systems.

The pruning item connects to the exactness slide: naive pruning changed the iterates and slowed convergence by a large factor, so a correct version must apply the skipped discounts lazily.



Return to the opening: the solver is a few lines of arithmetic, and its speed comes from structure exploited at the level of vectors, leaves, memory and processors. Then take questions.

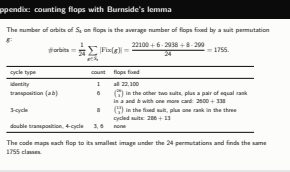
Appendix: the showdown sweep

The input is the own list sorted by strength q_h, the opponent list sorted by strength and the opponent reach r_-.

1. Set $\tau_- := 0$ and $s_c := 0$ for all cards c.
2. For each block of own hands with equal strength q:
   - 2.1 add every opponent hand j with $q_j < q$ to τ_- and to s_c for its two cards;
   - 2.2 for each own hand h of the block set $L(h) := \tau_- \cdot m_h - w_h$;
   - 2.3 add every opponent hand with $q_j = q$, and set $L^=(h) := \tau_- \cdot s_{h_1} - s_{h_2} + r_{j_-}$.
3. After the sweep, $m(h)$ is the fold formula with the final sums.

Each opponent hand is added once and each own hand is read twice, so the cost is $O(n + m)$.

The identical hand $\bar{h}$ cannot be strictly weaker than h, so the correction appears only in $L^=$.



Appendix: counting flops with Burnside's lemma

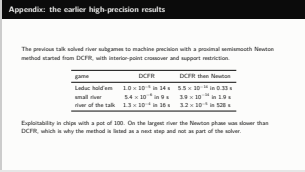
The number of orbits of S_4 on flops is the average number of flops fixed by a suit permutation g:

$$\#\text{orbits} = \frac{1}{|S_4|}\sum_{g\in S_4}|\mathrm{Fix}(g)| = \frac{22100 + 6\cdot 2938 + 8\cdot 299}{24} = 1755.$$

cycle type	count	flops fixed
identity	1	all $22\,100$
transposition $(a\,b)$	6	$\binom{13}{2}$ in the other two suits, plus a pair of equal rank in a and b with one more card: $2600 + 338$
3-cycle	8	$\binom{13}{3}$ in the fixed suit, plus one rank in the three cycled suits: $286 + 13$
double transposition, 4-cycle	3, 6	none

The code maps each flop to its smallest image under the 24 permutations and finds the same 1755 classes.

A flop has three cards, so a permutation that moves every suit cannot fix it unless its cards are permuted among themselves, which needs a cycle of length dividing the number of cards in the cycled suits. That rules out double transpositions and 4-cycles.


Appendix: the earlier high-precision results

The previous talk solved river subgames to machine precision with a proximal semismooth Newton method started from DCFR, with interior-point crossover and support restriction.

game	DCFR	DCFR then Newton
Leduc hold'em	1.0×10^{-4} in 14 s	5.5×10^{-16} in 0.33 s
small river	5.4×10^{-4} in 9 s	3.9×10^{-16} in 1.9 s
river of the talk	1.3×10^{-4} in 16 s	3.2×10^{-16} in 528 s

Exploitability in chips with a pot of 100. On the largest river the Newton phase was slower than DCFR, which is why the method is listed as a next step and not as part of the solver.

These are the numbers from `talk-newton.pdf`. The honest summary is that the finish is spectacular on small and medium games and not yet competitive on the largest river; the linear algebra of the dense leaf blocks inside the Newton systems is the bottleneck.